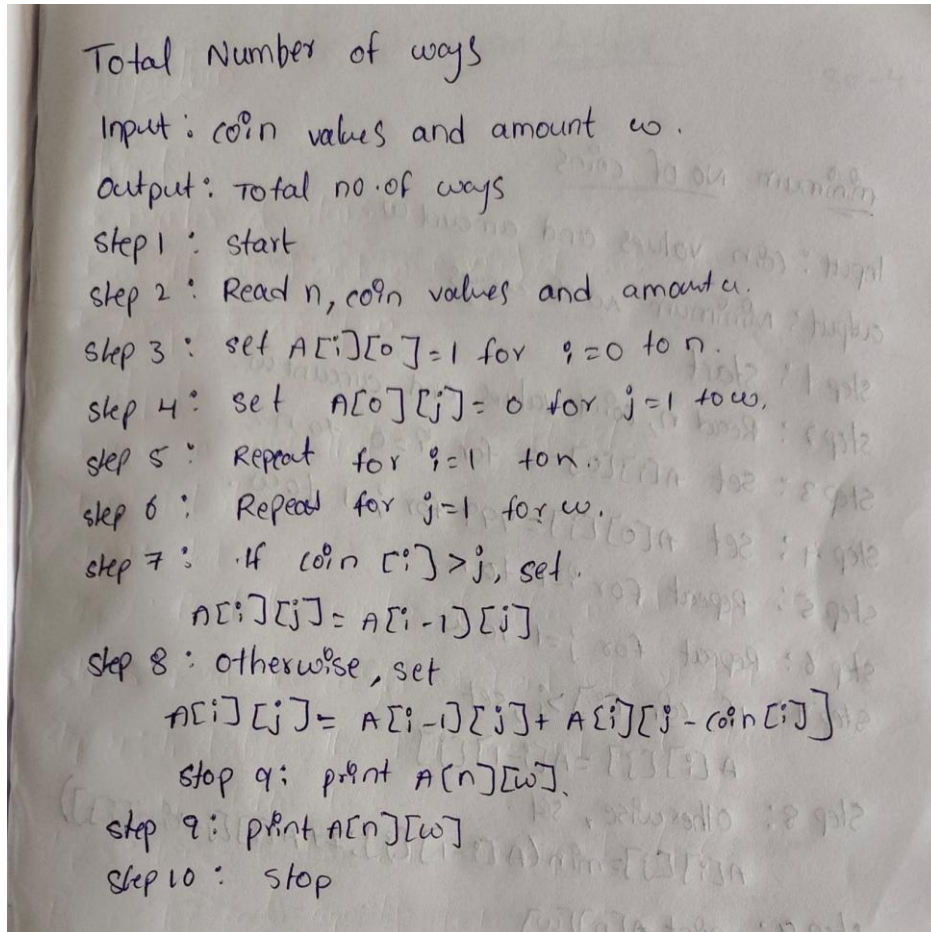


Practical- 7

Aim: To Implementation of making a change problem using dynamic programming.

Method-1: Total no of ways

Algorithm:



Total Number of ways

Input: coin values and amount w .

Output: Total no. of ways

step 1: start

step 2: Read n , coin values and amount w .

step 3: set $A[i][0] = 1$ for $i = 0$ to n .

step 4: set $A[0][j] = 0$ for $j = 1$ to w .

step 5: Repeat for $i = 1$ to n .

step 6: Repeat for $j = 1$ to w .

step 7: If $\text{coin}[i] > j$, set

$$A[i][j] = A[i-1][j]$$

step 8: otherwise, set

$$A[i][j] = A[i-1][j] + A[i][j - \text{coin}[i]]$$

step 9: print $A[n][w]$.

step 10: stop

Program:

```
#include <iostream>
using namespace std;
class ChangeMaking {
private:
    int coins[10];
    int n;
    int w;
    int A[10][100];
public:
```



```
void getInput() {
    cout << "Enter number of coin types : ";
    cin >> n;
    cout << "Enter coin values : ";
    for (int i = 1; i <= n; i++)
        cin >> coins[i];
    cout << "Enter amount : ";
    cin >> w;
}

void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 1;
    for (int j = 1; j <= w; j++)
        A[0][j] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = A[i - 1][j]
                    + A[i][j - coins[i]];
        }
    }
}

void printTable() {
    cout << "\nDP Table:\n";
    cout << "Coin ";
    for (int j = 0; j <= w; j++)
        cout << j << " ";
    cout << endl;
    for (int i = 0; i <= n; i++) {
        if (i == 0)
            cout << "0  ";
    }
}
```

```
else
    cout << coins[i] << " ";
for (int j = 0; j <= w; j++)
    cout << A[i][j] << " ";
cout << endl;
}
}
void showResult() {
    cout << "\nTotal Number of Ways = "
        << A[n][w] << endl;
}
};
int main() {
    ChangeMaking c;
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();
    return 0;
}
```

Output:

```
Enter number of coin types : 4
Enter coin values : 2
5
1
3
Enter amount : 10

DP Table:
Coin 0 1 2 3 4 5 6 7 8 9 10
0 1 0 0 0 0 0 0 0 0 0
2 1 0 1 0 1 0 1 0 1 0 1
5 1 0 1 0 1 1 1 1 1 1 2
1 1 1 2 2 3 4 5 6 7 8 10
3 1 1 2 3 4 6 8 10 13 16 20

Total Number of Ways = 20

...Program finished with exit code 0
Press ENTER to exit console.
```



Time complexity:

worst case:

```
for (int i=1) i < n; i++ {n
  for (int j=1; j <= w; j++) {
```

```
    if (coins[i] > j) {
```

```
        A[i][j] = A[i-1][j];
```

```
    }  
    else {
```

```
        A[i][j] = A[i-1][j] + (A[i][j] - coins[i]);
```

```
    }  
  }  
}
```

n = no of coins type, w = target amount.

$$T(n, w) = 1 + (n+1) + n + n + n(w+1) + nw + nw + nw.$$

$$= 4nw + 2n + 2.$$

$$= O(nw) \rightarrow \text{in worst case.}$$

Average case: The loops still run for every coin and every amount.

$$T(n, w) = O(nw)$$

Best case: The complete DP table is calculated. The algorithm does not stop early.

$$T(n, w) = O(nw)$$

Method-2: Minimum no of ways

Algorithm:

minimum no of coins
Input: coin values and amount w .
Output: minimum no of ways.
Step 1: Start
Step 2: Read n , coin values and amount w .
Step 3: Set $A[i][0] = 0$ for $i = 0$ to n .
Step 4: Set $A[0][j] = 999$ for $j = 1$ to w .
Step 5: Repeat for $i = 1$ to n .
Step 6: Repeat for $j = 1$ to w .
Step 7: If $\text{coin}[i] > j$, set
 $A[i][j] = A[i-1][j]$
Step 8: Otherwise, set
 $A[i][j] = \min(A[i-1][j], 1 + A[i][j - \text{coin}[i]])$
Step 9: Print $A[n][w]$
Step 10: Stop.

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;
class ChangeMaking {
private:
    int coins[10];
    int n, w;
    int A[100][100];
public:
    void getInput() {
```



```
cout << "Enter number of coin types: ";
cin >> n;
cout << "Enter coin values: ";
for (int i = 1; i <= n; i++){
    cin >> coins[i];
cout << "Enter amount: ";
cin >> w;
}

void solve() {
    for (int i = 0; i <= n; i++)
        A[i][0] = 0;
    for (int j = 1; j <= w; j++)
        A[0][j] = 999;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= w; j++) {
            if (coins[i] > j)
                A[i][j] = A[i - 1][j];
            else
                A[i][j] = min(A[i - 1][j],
                    1 + A[i][j - coins[i]]);
        }
    }
}

void printTable() {
    cout << "\nDP Table:\n";
    cout << "    ";
    for (int j = 0; j <= w; j++)
        cout << j << " ";
    cout << endl;
    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for (int j = 0; j <= w; j++) {
```



```
        if(A[i][j] == 999)
            cout << "0 ";
        else
            cout << A[i][j] << " ";
        cout << endl; } }

void showResult() {
    if(A[n][w] == 999)
        cout << "\nAmount cannot be formed.";
    else
        cout << "\nMinimum number of coins = " << A[n][w];
    }
};

int main() {
    ChangeMaking c;
    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();
    return 0;
}
```

Output:

```
Enter number of coin types: 4
Enter coin values: 3
7
2
5
Enter amount: 15

DP Table:
0 INF INF INF INF INF INF INF INF INF INF INF INF INF INF
0 INF INF 1 INF INF 2 INF INF 3 INF INF 4 INF INF 5
0 INF INF 1 INF INF 2 1 INF 3 2 INF 4 3 2 5
0 INF 1 1 2 2 2 1 3 2 2 3 3 3 2 4
0 INF 1 1 2 1 2 1 2 2 2 3 2 3 2 3

Minimum number of coins = 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Time complexity:

Time complexity:

```
for (int i = 1; i ≤ n; i++) {
```

```
    for (int j = 1; j ≤ w; j++) {
```

```
        if (coins[i] > j) {
```

```
            A[i][j] = A[i-1][j]
```

```
        }
```

```
    } else {
```

```
        A[i][j] = min[A[i-1][j], 1 + A[i][j - coins[i]]]
```

```
    }
```

```
}
```

$$T(n, w) = 1 + (n+1) + n + n(w+1) + nw + nw + nw$$

$$= nw + n + 2$$

Average case: $T(n, w) = O(nw) \rightarrow$ loop runs to every coin and every amount.

Best case: $T(n, w) = O(nw)$ all dp table cells are calculated because there is no break.

Conclusion: The Making Change problem was successfully implemented using Dynamic Programming. The algorithm stores the solutions of smaller subproblems in a DP table and uses them to find the required solution efficiently. It reduces repeated calculations and provides an efficient approach for finding the minimum number of coins or total number of ways to make a given amount.